

1. Weighted Median Consider n numbers $p_1, \dots, p_n$ with associated positive weights $w_1, \dots, w_n$ which satisfy that $\sum_{i=1}^n w_i = 1$. We say that p_k is a *weighted median* of the given numbers if $\sum_{p_i < p_k} w_i < \frac{1}{2}$ and $\sum_{p_i > p_k} w_i \leq \frac{1}{2}$. Describe an algorithm which finds a weighted median of given numbers in time $O(n)$. The input numbers are given in an array which is not sorted.

We can modify Blum's algorithm for finding k -th smallest element in a sequence in linear time—an idea similar to quick-sort. We pick an arbitrary pivot p and split the input sequence accordingly. We do not use weights to pick the pivot, only to decide which set should we continue with: L , E or G .

Algorithm `weighted_quick_select( $P, \varepsilon$ )`

```

Pick an arbitrary pivot  $(p^*, w^*) \in P$ 
 $L \leftarrow \{(p_i, w_i) : p_i < p^*\}_{i=1}^n$ 
 $E \leftarrow \{(p_i, w_i) : p_i = p^*\}_{i=1}^n$ 
 $G \leftarrow \{(p_i, w_i) : p_i > p^*\}_{i=1}^n$ 
 $w_L = \sum_{(p,w) \in L} w$ 
 $w_E = \sum_{(p,w) \in E} w$ 
 $w_G = \sum_{(p,w) \in G} w$ 
if  $w_L \geq \varepsilon$  then
    return weighted_quick_select( $L, \varepsilon$ )
else if  $w_L + w_E < \varepsilon$  then
    return weighted_quick_select( $G, \varepsilon$ )
else
    for  $(p, w) \in E$  do
         $w_L \leftarrow w_L + w$ 
        if  $w_L > \varepsilon$  then
            return  $p$ 
        end if
    end for
end if

```

2. Post Office Placement We are given n points $P = \{p_1, \dots, p_n\}$ which have associated positive weights $w_1, \dots, w_n$ satisfying $\sum_{i=1}^n w_i = 1$ and a distance function $d(p_i, p_j) > 0$ which gives the distance between any two points p_i and p_j . We want to place the post office at a point p^* which minimizes the value of $\sum_{i=1}^n w_i d(p^*, p_i)$.

- Argue that, if $P \in \mathbb{R}^n$ and $d(p_i, p_j) = |p_i - p_j|$, then the optimal choice for p^* is the weighted median of the points.
- What is an optimal choice of p^* if $P \in \mathbb{R}^{n \times 2}$ and $d(p_i, p_j) = |p_{i1} - p_{j1}| + |p_{i2} - p_{j2}|$.

- Let $c(p) = \sum_{i=1}^n w_i d(p, p_i)$. Since $d(p, p_i) = |p - p_i|$, we have

$$c(p) = \sum_{p_i < p} w_i (p - p_i) + \sum_{p_i > p} w_i (p_i - p)$$

Let us assume for simplicity that the points are sorted (i.e., that $p_1 < p_2 < \dots < p_n$) and denote p_m the weighted median. Assume p^* is a point which minimizes $c(p^*)$ and that, for the sake of contradiction, $p^* < p_m$ —the case when p^* is bigger is symmetrical and we will be left with $p^* = p_m$.

Assume first that p^* is not equal to any point p_k . In particular, assume that there exists an index k such that $p_{k-1} < p^* < p_k \leq p_m$ and let $\varepsilon = p_k - p^*$.

12th tutorial (continued)

Assuming (without loss of generality) that P is sorted, we find that

$$\begin{aligned}
 c(p_k) &= \sum_{i=1}^{k-1} w_i (p_k - p_i) + \sum_{i=k+1}^n w_i (p_i - p_k) \\
 &= \sum_{i=1}^{k-1} w_i (p^* + \varepsilon - p_i) + \sum_{i=k+1}^n w_i (p_i - p^* - \varepsilon) \\
 &= \left(\sum_{i=1}^{k-1} w_i (p^* - p_i) + \sum_{i=k+1}^n w_i (p_i - p^*) \right) + \left(\sum_{i=1}^{k-1} w_i \varepsilon - \sum_{i=k+1}^n w_i \varepsilon \right) \\
 &< \left(\sum_{i=1}^{k-1} w_i (p^* - p_i) + \sum_{i=k+1}^n w_i (p_i - p^*) + w_k (p_k - p^*) \right) + \varepsilon \left(\sum_{i=1}^{k-1} w_i - \sum_{i=k+1}^n w_i \right) \\
 &= c(p^*) + \varepsilon \left(\sum_{i=1}^{k-1} w_i - \sum_{i=k+1}^n w_i \right) \\
 &\leq c(p^*)
 \end{aligned}$$

Where the first inequality holds because we included a positive term $w_k (p_k - p^*)$ and the second inequality holds because $p_k \leq p_m$ (where p_m is the weighted median). This stands in contradiction to the optimality of p^* .

Let us instead assume that $p^* = p_k$ for some $k < m$. Assuming again that the points are sorted so that $\varepsilon = p_{k+1} - p_k > 0$, we find that

$$\begin{aligned}
 c(p_{k+1}) &= \sum_{i=1}^{k-1} w_i (p_{k+1} - p_i) + w_k (p_{k+1} - p_k) + \sum_{i=k+2}^n w_i (p_i - p_{k+1}) \\
 &< \left(\sum_{i=1}^{k-1} w_i (p_k + \varepsilon - p_i) + w_k \varepsilon + \sum_{i=k+2}^n w_i (p_i - p_k - \varepsilon) \right) + w_{k+1} (p_{k+1} - p_k) \\
 &= \left(\sum_{i=1}^{k-1} w_i (p_k - p_i) + \sum_{i=k+1}^n w_i (p_i - p_k) \right) + \varepsilon \left(\sum_{i=1}^k w_i - \sum_{i=k+2}^n w_i \right) \\
 &= c(p_k) + \varepsilon \left(\sum_{i=1}^k w_i - \sum_{i=k+2}^n w_i \right) \\
 &\leq c(p_k)
 \end{aligned}$$

Where the first inequality follows from adding a positive term $w_k (p_{k+1} - p_k)$ and the last inequality follows from the fact that $p_{k+1} \leq p_m$ (where p_m is the weighted median). This also stands in contradiction with the optimality of p . $\square$

- (b) We pick x to be the weighted median of the x -coordinates and y to be the weighted median of the y -coordinates. We then return (x, y) . It is interesting to note that position (x, y) might not be present in the input data.

3. Nadiria In a previous life, you worked as a cashier in the lost Antarctic colony of Nadiria, spending the better part of your day giving change to your customers. Because paper is a very rare and valuable resource in Antarctica, cashiers were required by law to use the fewest bills possible whenever they gave change. Thanks to the numerological predilections of one of its founders, the currency of Nadiria was available in the following denominations: \$1, \$4, \$7, \$13, \$28, \$52, \$91, and \$365.

- (a) Describe and analyze a recursive algorithm that computes, given an integer k , the minimum number of bills needed to make k Dream-Dollars. (Don't worry about making your algorithm fast; just make sure it's correct.)
- (b) Describe a dynamic programming algorithm that computes, given an integer k , the minimum number of bills needed to make k Dream-Dollars. (This one needs to be fast.)

12th tutorial (continued)

Let us denote the set of denominations as $D = \{1, 4, 7, 13, 28, 52, 91, 356\}$. For a given value k , let us also denote $N(k)$ the minimum number of bills needed to make k .

- (a) Consider the following recurrence:

$$N(k) = \begin{cases} 1 + \min\{N(k-d) : d \in D \wedge k \geq d\} & k > 0 \\ 0 & k = 0 \end{cases}$$

The correctness of this recurrence can be argued as follows: If $k = 0$, then we need no bills, so clearly $N(0) = 0$. If $k > 0$, we will need at least one bill $d \in D$. The bill we use cannot have a denomination bigger than k , so $k \geq d$. After we use the bill d , we need to optimally split the remaining amount $k - d$; thus, $N(k - d) + 1$.

Algorithm recursive_N(k)

```

if  $k = 0$  then
    return 0
else
     $n \leftarrow k$ 
    for  $d \in D$  do
        if  $d \leq k$  then
             $n \leftarrow \min(n, 1 + \text{recursive\_N}(k - d))$ 
        end if
    end for
    return  $n$ 
end if

```

- (b) One way to fix function `recursive_N` is to use memoization. This means adding a cache used to store the results of the recursive calls. When the function is called second time with the same parameter, the cached value is used and the value needs not to be recomputed. This approach is described in function `cached_N`. Each value is computed at most once, so the complexity of the algorithm is now $O(|D| \cdot k)$.

Algorithm cached_N(k)

```

if  $k = 0$  then
    return 0
else if  $C(k) \neq \text{None}$  then
    return  $C(k)$ 
else
     $C(k) \leftarrow k$ 
    for  $d \in D$  do
        if  $d \leq k$  then
             $C(k) \leftarrow \min(C(k), 1 + \text{recursive\_N}(k - d))$ 
        end if
    end for
    return  $C(k)$ 
end if

```

We may also observe that to evaluate $N(k)$, we only need values of N on indices smaller than k . This suggests filling an array with values of N in indices considered in increasing order starting with index 0. This approach is described in function `tabulated_N`. The complexity of this algorithm is $O(|D| \cdot k)$.

Algorithm dynamic_N(k)

```

 $N \leftarrow$  empty array of length  $k + 1$ 
 $N(0) \leftarrow 0$ 
for  $i \leftarrow 1$  to  $k$  do
     $N(i) \leftarrow i$ 
    for  $d \in D$  do
        if  $d \leq i$  then
             $N(i) \leftarrow \min(N(i), 1 + N(i - d))$ 
        end if
    end for
end for
return  $N(k)$ 

```

4. Library Say that we have a sequence of n books in alphabetical order—each with a width w_i and a height h_i —and a single library cabinet with several shelves of fixed width W but mutable height. We want to arrange the books so as to minimize the total height of the cabinet while preserving the alphabetical ordering.

The minimal height $H(i)$ of the cabinet with books 1 through i can be expressed recursively:

$$H(i) = \min \left\{ H(j-1) + \max\{h_k\}_{k=j}^i : j \in [1, i], \sum_{k=1}^i w_k \leq W \right\}, \quad H(0) = 0$$

where j is the index of the first book on the last shelf.

Since the book's widths are not fixed, we cannot put bounds on the interior maximization and computing it may cost $\mathcal{O}(i)$. Finding $H(n)$ recursively may then take up to $\mathcal{O}(n^2)$.

5. Broadcasting in a Tree Suppose we need to broadcast a message to all the nodes in a rooted binary tree. Initially, only the root node knows the message. In a single round, any node that knows the message can forward it to at most one of its children. Design an algorithm to compute the minimum number of rounds required to broadcast the message to all nodes.

Let us consider a binary tree $T = (V, E)$ with root r . Let us denote $B(v)$ the minimum number of nodes needed to broadcast the message from v to all nodes in the subtree rooted at v . The value we are interested in is then $B(r)$. If v is a leaf, then we need no additional round, so $B(v) = 0$ in this case. Let us now assume that v has two child nodes v_1 and v_2 .

- If $B(v_1) < B(v_2)$, then it makes sense to first pass the message to v_2 (in 1 round), then we pass the message from v to v_1 . At the same time v_2 starts broadcasting into its subtree. After $B(v_1) + 1$ rounds, all nodes in the subtree of v_1 get the message and after $B(v_2) + 1$ all nodes in the subtree of v_2 get the message. We thus have $B(v) = B(v_2) + 1$.
- If $B(v_1) > B(v_2)$, we have $B(v) = B(v_1) + 1$ by a symmetric argument.
- If $B(v_1) = B(v_2)$, then we need $2 + B(v_1) = 2 + B(v_2)$ rounds. This can be seen as follows. In the first round we pass the message to one of the child nodes, say to v_1 (the case with v_2 is symmetrical). In the second round, we pass the message to v_2 , but then we still need $B(v_2)$ rounds to broadcast the message in the subtree of v_2 .

We can use the above case analysis to determine the values of $B[v]$ for every node $v \in V$ in linear time.